

Propuesta de Sistema y Pipeline de Datos para la Encuesta NAE

1. Visión general

Se propone implementar un sistema automatizado que conecte la encuesta de Google Forms con una base de datos centralizada, permitiendo capturar, validar, normalizar, almacenar y explotar analíticamente la información recopilada.

El sistema debe evitar procesos manuales y garantizar que cada respuesta capturada pueda ser utilizada posteriormente en reportes, tableros, fichas territoriales y análisis institucionales.

2. Arquitectura propuesta

Flujo general

Google Forms
↓
Google Sheets
↓
Google Apps Script
↓
API de integración
↓
Base de datos PostgreSQL
↓
Procesos ETL
↓
Modelo analítico / Data Mart
↓
Power BI / Metabase / Portal Web

3. Componentes del sistema

3.1 Google Forms

Responsable de la captura inicial de datos.

Funciones:

- Presentar la encuesta.
- Validar campos obligatorios.
- Registrar fecha y hora.

- Enviar respuestas a Google Sheets.
-

3.2 Google Sheets como staging inicial

La hoja de cálculo funcionará como primer repositorio de captura.

Funciones:

- Conservar la respuesta original.
 - Servir como respaldo inicial.
 - Activar el proceso automático de envío.
-

3.3 Google Apps Script

Se implementará un script asociado a la hoja de respuestas.

Funciones:

- Ejecutarse automáticamente al recibir una nueva respuesta.
- Leer la fila recién ingresada.
- Convertirla en JSON.
- Enviarla a una API mediante HTTP POST.
- Registrar errores de envío.

Ejemplo conceptual:

```
onFormSubmit(evento)
  leer respuesta
  construir JSON
  enviar a API
  registrar resultado
```

3.4 API de integración

La API será el punto de entrada seguro hacia la plataforma.

Tecnología recomendada:

- FastAPI con Python.

Funciones:

- Recibir respuestas desde Apps Script.
- Validar token de seguridad.
- Guardar respuesta cruda en staging.

- Normalizar datos.
- Insertar información en tablas operacionales.
- Registrar errores e incidencias.

Endpoints iniciales:

```
POST /api/v1/respuestas
GET /api/v1/salud
GET /api/v1/catalogos/provincias
GET /api/v1/catalogos/municipios
```

3.5 Base de datos PostgreSQL

Se recomienda usar PostgreSQL como base principal.

Capas internas:

```
raw
staging
operacional
analitica
```

4. Diseño del pipeline

Pipeline 1: Captura en tiempo real

Objetivo:

Registrar automáticamente cada respuesta nueva.

Flujo:

```
Nueva respuesta en Google Forms
→ Google Sheets
→ Trigger Apps Script
→ API
→ raw.respuestas_formulario
```

Resultado:

Ninguna respuesta se pierde, aunque todavía no esté normalizada.

Pipeline 2: Validación y normalización

Objetivo:

Convertir el dato capturado en dato estructurado.

Procesos:

- Validar campos obligatorios.
- Normalizar provincia.
- Normalizar municipio.
- Normalizar tipo de entidad.
- Separar preguntas multiselección.
- Identificar respuestas "Otro".
- Separar textos abiertos.
- Marcar respuestas con errores.

Resultado:

Datos cargados en modelo operacional.

Pipeline 3: Carga operacional

Objetivo:

Guardar datos limpios en tablas relacionales.

Tablas principales:

```
operacional.provincia
operacional.municipio
operacional.tipo_entidad
operacional.entidad
operacional.encuesta
operacional.respuesta
operacional.respuesta_tema_formacion
operacional.respuesta_institucion_participante
operacional.respuesta_limitacion
operacional.respuesta_texto_abierto
```

Pipeline 4: Construcción del modelo analítico

Objetivo:

Actualizar el esquema estrella para análisis.

Tablas:

```
analitica.F_RespuestaEncuesta  
analitica.D_Fecha  
analitica.D_Provincia  
analitica.D_Municipio  
analitica.D_Entidad  
analitica.D_TipoEntidad  
analitica.D_Informante  
analitica.D_Encuesta  
analitica.D_CatalogoRespuesta  
analitica.D_TemaFormacion  
analitica.D_Limitacion  
analitica.B_Respuesta_TemaFormacion  
analitica.B_Respuesta_InstitucionParticipante  
analitica.B_Respuesta_Limitacion
```

Pipeline 5: Publicación para visualización

Objetivo:

Exponer datos listos para consumo.

Productos:

```
vista_resumen_municipal  
vista_resumen_provincial  
vista_temas_prioritarios  
vista_limitaciones  
vista_infraestructura  
vista_articulacion  
vista_genero
```

Consumidores:

- Power BI.
- Metabase.
- Portal web.
- API pública o institucional.

5. Automatización recomendada

Opción MVP

Automatización ligera:

Google Apps Script → API → PostgreSQL

Ventajas:

- Rápida implementación.
- Bajo costo.
- Suficiente para el volumen inicial.

Opción robusta

Automatización con orquestador:

Google Apps Script → API → PostgreSQL → Airflow / Prefect → Data Mart

Ventajas:

- Mejor control de procesos.
- Reintentos automáticos.
- Monitoreo.
- Ejecución programada.
- Escalabilidad.

6. Propuesta tecnológica

Captura

Google Forms + Google Sheets.

Automatización

Google Apps Script.

API

FastAPI.

Base de datos

PostgreSQL.

ETL / Orquestación

Fase inicial:

- SQL programado.
- Jobs internos.

Fase avanzada:

- Prefect o Airflow.

Visualización

- Power BI.
- Metabase.
- Looker Studio.

Despliegue

Opciones:

- Servidor institucional.
- VPS.
- Docker.
- Supabase.
- Railway.
- Render.
- Google Cloud Run.

7. Seguridad

Medidas mínimas:

- Token secreto entre Apps Script y API.
 - HTTPS obligatorio.
 - Control de usuarios para dashboards.
 - Logs de carga.
 - Copias de seguridad.
 - Separación entre datos crudos y datos publicados.
 - Minimización de datos personales.
-

8. Control de calidad de datos

El sistema debe validar:

- Provincia existente.
- Municipio válido.
- Municipio perteneciente a la provincia.
- Tipo de entidad válido.
- Campos obligatorios.
- Respuestas multiselección válidas.
- Duplicados potenciales.
- Respuestas incompletas.
- Valores "No sabe" diferenciados de ausencia de dato.

Estados sugeridos:

```
recibida
validada
con_observaciones
rechazada
corregida
publicada
```

9. Monitoreo del pipeline

Se recomienda crear una tabla de control:

```
control.ejecucion_pipeline
```

Campos:

```
id
fecha_inicio
fecha_fin
tipo_pipeline
estado
registros_procesados
registros_error
mensaje_error
```

También una tabla de errores:


```
control.errores_carga
```

Campos:

```
id
respuesta_origen_id
campo
valor_recibido
tipo_error
descripcion
fecha_error
estado_correccion
```

10. Estrategia de implementación por fases

Fase 1: MVP funcional

Duración estimada: 1 a 2 semanas.

Entregables:

- Formulario funcionando.
- Google Sheets conectado.
- Apps Script con trigger.
- API básica.
- PostgreSQL.
- Tabla raw de respuestas.
- Logs básicos.

Fase 2: Modelo operacional

Duración estimada: 2 a 3 semanas.

Entregables:

- Tablas operacionales.
 - Catálogos territoriales.
 - Catálogos de respuestas.
 - Separación de multiselecciones.
 - Normalización de entidades.
 - Validaciones básicas.
-

Fase 3: Modelo analítico

Duración estimada: 2 semanas.

Entregables:

- Esquema estrella.
 - Vistas analíticas.
 - Indicadores iniciales.
 - Preparación para Power BI.
-

Fase 4: Visualización

Duración estimada: 2 a 3 semanas.

Entregables:

- Dashboard nacional.
 - Dashboard provincial.
 - Ficha municipal.
 - Reportes exportables.
-

Fase 5: Gobernanza e interoperabilidad

Duración estimada: continua.

Entregables:

- Roles de usuario.
 - Documentación técnica.
 - API de consulta.
 - Manual de operación.
 - Procedimiento de respaldo.
 - Política de calidad de datos.
-

11. Resultado esperado

Al finalizar la implementación, se contará con un sistema automatizado capaz de:

- Capturar respuestas en tiempo real.
- Guardar datos originales.
- Normalizar información.
- Alimentar una base operacional.
- Construir un modelo analítico.
- Generar dashboards e indicadores.

- Integrarse con otros sistemas.
 - Escalar a nuevas encuestas o levantamientos.
-

12. Conclusión

La implementación del sistema y pipelines propuestos permitirá convertir la encuesta en una infraestructura de datos sostenible.

El valor principal no estará únicamente en almacenar respuestas, sino en transformar la información capturada en conocimiento territorial útil para la toma de decisiones, la planificación de acciones formativas y el fortalecimiento de los Nuevos Actores Económicos.